CSCC 全国大学生计算机系统能力大赛

GPUC 实验复现指南

GPUC——面向透明存储压缩的 GPU 解压卸载原型

学校	西安交通大学
队伍名称	404notfound
队伍编号	T2026106989910286
选题编号	proj35：基于 GPU 的服务资源卸载技术探索
队员姓名	朱嘉乐（队长）、贾茹、边程曦
指导老师	聂世强、刘松

2026 年 8 月 12 日

目录

1. 环境准备 1
1.1 基本要求 1
1.2 编译与基础测试 1
2. 基础功能演示 1
3. 推荐复现的五组实验 2
3.1 核心 CPU/GPU 卸载 2
3.2 SQLite 并行业务 2
3.3 fio 多线程正确性与退出稳定性 3
3.4 Vdbench 端到端 I/O 3
3.5 长期稳定性与故障恢复 3
4. 可选资源实验：Intel PCM 4
5. 结果整理 4
6. 常见问题 5
6.1 Hook 没有生效 5
6.2 GPU 初始化失败 5
6.3 fio 完成后不退出 5
6.4 Daemon 报空间不足 5

1. 环境准备

1.1 基本要求

- x86_64 Linux；
- NVIDIA GPU，已验证 Tesla V100 与 GeForce RTX 4090；
- NVIDIA 驱动、CUDA Toolkit、GCC/G++、GNU Make、fio 和 Python 3；
- Vdbench 与 Intel PCM 仅在运行对应实验时准备。

Ubuntu/Debian 基础依赖：

```
sudo apt-get update
sudo apt-get install -y build-essential fio python3 bc xxd sysstat
```

CUDA 驱动与 Toolkit 应根据实验机器和 NVIDIA 官方说明安装。先检查环境：

```
nvidia-smi
nvcc --version
bash scripts/check_env.sh
```

1.2 编译与基础测试

```
bash scripts/setup_dependencies.sh
make clean
make all
make test
```

主要产物：

文件	用途
bin/libgpuhook.so	GPU 透明 I/O Hook
bin/libcpuhook.so	CPU 对照 Hook
bin/gpu_compress_daemon	GPU 离线压缩 Daemon
bin/cpu_compress_daemon	CPU 离线压缩 Daemon

2. 基础功能演示

以下命令完成“写入 RAW—后台压缩—透明读取”：

```
mkdir -p /tmp/gpuc_demo
```

```
LD_PRELOAD="$PWD/bin/libgpuhook.so" GPUCOMPRESS_QUIET=1 \
  dd if=/dev/zero of=/tmp/gpuc_demo/data.bin bs=1M count=64 status=none
GPUC_ALGO=lz4 "$PWD/bin/gpu_compress_daemon" /tmp/gpuc_demo
LD_PRELOAD="$PWD/bin/libgpuhook.so" GPUCOMPRESS_QUIET=1 \
  dd if=/tmp/gpuc_demo/data.bin bs=1M status=none | sha256sum
```

也可以运行一键演示：

```
bash scripts/demo.sh
```

3. 推荐复现的五组实验

进入项目根目录并设置 GPU。多卡机器需要把编号改为目标设备：

```
cd /home/dell/zhujiale/project3136859-386240
export GPUC_PHYSICAL_GPU=1
export CUDA_VISIBLE_DEVICES=1
export RUN_TAG="reproduce_$(date -u +%Y%m%dT%H%M%SZ)"
```

3.1 核心 CPU/GPU 卸载

用途：验证相同解压吞吐下，GPU 是否减少主机 CPU 竞争并保护业务吞吐。

```
GPUC_RESULT_DIR="$PWD/results/local/resource_${RUN_TAG}" \
GPUC_RUNTIME=10 GPUC_REPEATS=5 \
GPUC_BUSINESS_WORKERS=4 GPUC_TARGET_MIB_S=4096 \
GPUC_DECOMP_CPU=0 GPUC_WORKER_CPU=0 \
bash tests/system_resource_profile.sh
```

重点查看：

```
cat "results/local/resource_${RUN_TAG}/summary.csv"
```

比较 CPU/GPU 的 business_ops_s、decomp_mib_s、submit_cpu_sec、 worker_cpu_sec 和 maxrss_kib。只有 CPU/GPU 实际解压吞吐接近时，才计算卸载收益。

3.2 SQLite 并行业务

用途：验证释放 CPU 后，真实数据库业务的 QPS 和尾延迟是否改善。

```
GPUC_RESULT_DIR="$PWD/results/local/sqlite_${RUN_TAG}" \
GPUC_RUNTIME=5 GPUC_REPEATS=5 GPUC_BUSINESS_WORKERS=4 \
GPUC_SQLITE_ROWS=100000 GPUC_SQLITE_SPAN=1024 \
GPUC_TARGET_MIB_S=4096 GPUC_DECOMP_CPU=0 GPUC_WORKER_CPU=0 \
bash tests/sqlite_offload_business.sh
```

重点查看：

```
cat "results/local/sqlite_${RUN_TAG}/summary.csv"
```

比较 Baseline、CPU、GPU 的 QPS、p99 和解压进程 CPU 时间。

3.3 fio 多线程正确性与退出稳定性

用途：验证 SGLZ/LZ4 在不同块大小和线程数下无错误、无退出卡死、SHA 一致。

```
GPUC_RESULT_DIR="$PWD/results/local/fio_threads_${RUN_TAG}" \
GPUC_SIZE=64M GPUC_RUNTIME=3 GPUC_REPEATS=5 \
GPUC_BS_LIST='4k 16k 128k' GPUC_THREAD_LIST='1 4' \
bash tests/fio_compressed_thread_regression.sh
```

重点查看：

```
cat "results/local/fio_threads_${RUN_TAG}/summary.csv"
```

正式配置共 60 个样本。验收条件是 fio error=0、无超时且逻辑 SHA-256 一致。

3.4 Vdbench 端到端 I/O

用途：使用赛题明确提到的 Vdbench，验证四条读取路径及混合读写路径。

Vdbench 需要使用者自行接受 Oracle 许可证并放入 third_party/manual/vdbench50407/ 。安装 Java 后先自检：

```
export GPUC_JAVA_HOME="$(dirname "$(dirname "$(readlink -f "$(command -
v java)")")")"
third_party/manual/vdbench50407/vdbench -t
```

正式运行：

```
GPUC_RESULT_DIR="$PWD/results/local/vdbench_${RUN_TAG}" \
GPUC_JAVA_HOME="$GPUC_JAVA_HOME" \
GPUC_SIZE=256m GPUC_RUNTIME=10 GPUC_WARMUP=2 \
GPUC_REPEATS=5 GPUC_THREADS=4 \
GPUC_READ_SCENARIOS='baseline hook_raw sglz lz4' \
GPUC_READ_WORKLOADS='seq_read random_read' GPUC_RUN_MIXED=1 \
bash tests/vdbench_benchmark.sh
```

重点查看：

```
cat "results/local/vdbench_${RUN_TAG}/summary.csv"
```

脚本保存 Vdbench HTML、逐轮 CSV、环境信息和逻辑 SHA-256。

COMPRESSED 是只读对象， 因此 70:30 混合读写只运行 Baseline 与 Hook RAW。

3.5 长期稳定性与故障恢复

正式一小时持续读取：

```
GPUC_RESULT_DIR="$PWD/results/local/stability_1h_${RUN_TAG}" \
```

```
GPUC_RUNTIME=3600 GPUC_REPEATS=1 \
GPUC_SIZE=512M GPUC_BS=128k GPUC_JOBS=4 \
bash tests/long_term_stability.sh
```

只做快速检查时，可把 GPUC_RUNTIME 改为 60。正式验收检查 fio error、每秒带宽 样本数量以及运行前后 SHA-256：

```
cat "results/local/stability_1h_${RUN_TAG}/summary.csv"
cat "results/local/stability_1h_${RUN_TAG}/environment.txt"
```

故障恢复测试不需要 root：

```
GPUC_RESULT_DIR="$PWD/results/local/fault_${RUN_TAG}" \
bash tests/fault_recovery.sh
cat "results/local/fault_${RUN_TAG}/summary.csv"
```

验收条件为 ENOSPC、SIGKILL、重启恢复、GPU 不可见 fallback 和 /dev/full 五项 全部 PASS。

4. 可选资源实验：Intel PCM

只有机器已经安装 PCM 且允许访问 MSR/uncore 计数器时运行：

```
test -x third_party/manual/pcm/build/bin/pcm-memory \
  && echo PCM_READY || echo PCM_MISSING
```

若输出 PCM_READY：

```
sudo env \
  PATH="$PATH" GPUC_PHYSICAL_GPU="$GPUC_PHYSICAL_GPU" \
  CUDA_VISIBLE_DEVICES="$CUDA_VISIBLE_DEVICES" \
  GPUC_RESULT_DIR="$PWD/results/local/pcm_${RUN_TAG}" \
  GPUC_RUNTIME=10 GPUC_REPEATS=5 GPUC_TARGET_MIB_S=4096 \
  GPUC_COMPRESSIBLE_PCT=100 GPUC_WORKSET_MIB=512 \
  GPUC_BATCH_BLOCKS=128 GPUC_DECOMP_CPU=0 GPUC_WORKER_CPU=0 \
  bash tests/pcm_dram_offload.sh
```

先检查 CPU/GPU 的实际 decomp_mib_s。只有两条路径实际吞吐相近，才能把 DRAM 总带宽差解释为同吞吐卸载收益；否则只报告各自实际吞吐和 PCM 采样结果。

5. 结果整理

本轮所有输出目录：

```
find results/local -maxdepth 1 -type d -name "*${RUN_TAG}" -print
```

每个实验目录至少保留：

- environment.txt：硬件、软件和运行参数；
- runs.csv：逐轮原始指标；
- summary.csv：均值和置信区间；
- fio JSON、Vdbench HTML 或 PCM CSV 等工具原始输出；
- 正确性实验的逻辑 SHA-256。

正式材料优先引用 summary.csv，出现异常时再回看逐轮日志。完整历史实验索引见 `results/final/README.md`。

6. 常见问题

6.1 Hook 没有生效

- 确认 LD_PRELOAD 使用正确的绝对路径；
- 使用 ldd 确认目标程序是动态链接；
- 静态链接、直接 syscall 和应用自行提交的 io_uring 不在当前覆盖范围。

6.2 GPU 初始化失败

```
nvidia-smi
nvcc --version
```

确认 GPUC_PHYSICAL_GPU 与 CUDA_VISIBLE_DEVICES 指向同一块目标 GPU。GPU 不可用时，可使用 CPU Hook 验证格式和回退逻辑。

6.3 fio 完成后不退出

运行 tests/fio_compressed_thread_regression.sh 进行专项检查。脚本带有超时保护， 并会同时验证错误数和逻辑 SHA-256。

6.4 Daemon 报空间不足

Daemon 需要在目标文件同目录创建临时文件并原子替换，运行前应保证该文件系统有 足够可用空间。